

Project A3: Approximate Membership Filters (XOR vs Cuckoo vs Quotient)

1 System and Experimental Control

1.1 Hardware Platform

Experiments ran on one machine:

- **CPU:** 12th Gen Intel Core i7-1260P (Alder Lake)
- **Cores / Threads:** 12 cores (4P + 8E), 16 hardware threads
- **Clock frequency:** 400 MHz min, up to 4.7 GHz turbo
- **Vector ISA:** AVX2 with FMA
- **Memory:** 16 GiB DDR

The system uses one NUMA node. Runs used a lightly loaded system to reduce noise.

1.2 Software Environment

- **OS:** Ubuntu Linux 22.04 (kernel 6.8.0)
- **Compiler:** GCC 11.4.0
- **Compiler flags:**

```
-O3 -mavx2 -mfma -ffast-math -fopenmp -std=c++17
```

- **Perf counters:** Linux perf 6.8.12

Two binaries were used to separate effects: a scalar build (vectorization disabled) and a SIMD build (AVX2/FMA enabled).

Variability controls.

- CPU pinning used for benchmarks (fixed CPU set per run).
- CPU frequency kept stable (governor fixed; turbo policy unchanged during runs).
- Transparent Huge Pages disabled:
`echo never | sudo tee /sys/kernel/mm/transparent_hugepage/enabled.`
- Background activity minimized (AC power, no heavy interactive loads).
- Each data point reported as mean $\pm$ std over repeated runs (error bars in plots).

2 Reproducibility

Repository layout. `src/` (filters + benchmark), `scripts/` (sweeps), `plots/` (plotting), (exported figures).

Build.

```
# from repo root
make -j
```

Run full sweep (generates CSV + figures).

```
./scripts/run_sweep.sh
python3 plots/make_plots.py --in results/ --out
```

Perf collection (example).

```
perf stat -e cycles,instructions,branches,branch-misses,cache-references,
    cache-misses,\
dTLB-load-misses,L1-dcache-load-misses,LLC-load-misses \
    ./benchmark --filter cuckoo --dist uniform --neg 0.5 --threads 1
```

Experiment matrix

- Structures: blocked Bloom (baseline), XOR (static), Cuckoo (dynamic), Quotient (dynamic).
- Distributions: sequential and uniform keys.
- Accuracy/space: bits-per-entry (BPE) vs achieved FPR across multiple configurations.
- Workloads: read-only, read-mostly, balanced, insert/delete (where supported).
- Negative share: 0%, 50%, 90% (read-only stress).
- Load factor sweep for online updates (Cuckoo/Quotient).
- Thread scaling: 1, 2, 4, 8, 12 threads.
- Perf counters used to connect results to cache/TLB/branch behavior.

3 Results and Discussion

3.1 Space vs Accuracy (Bits per Entry vs Achieved FPR)

Figures 1 and 2 show bits per entry (BPE), including metadata, versus achieved false positive rate (FPR) for blocked Bloom, XOR, Cuckoo, and Quotient filters at three target FPRs (0.05, 0.01, 0.001) under sequential and uniform keys.

Blocked Bloom consistently uses the lowest BPE because it stores only a bit array and hash state. However, its achieved FPR deviates more from strict targets at low FPRs due to hash collisions and block-level correlation. This behavior is expected from Bloom theory when the number of hash functions is constrained and memory is blocked for cache locality.

XOR filters achieve lower achieved FPR at comparable or slightly higher BPE. This is explained by their use of fixed fingerprints and a perfect-hash-like construction that eliminates multi-probe behavior. Once built, lookups require only three memory accesses, leading to tight FPR control. The cost is static construction: rebuilds are required if keys change.

Cuckoo filters occupy more BPE than Bloom due to bucket metadata and fingerprints but achieve lower FPR at similar space because fingerprints are stored explicitly and verified on lookup. Quotient filters exhibit the highest BPE because they store remainder bits plus metadata for run

boundaries and cluster tracking. The higher space cost directly reflects this additional structural state.

Uniform keys show slightly worse space efficiency than sequential keys for Cuckoo and Quotient filters because hashing spreads clusters less predictably, increasing average probe and metadata overhead.

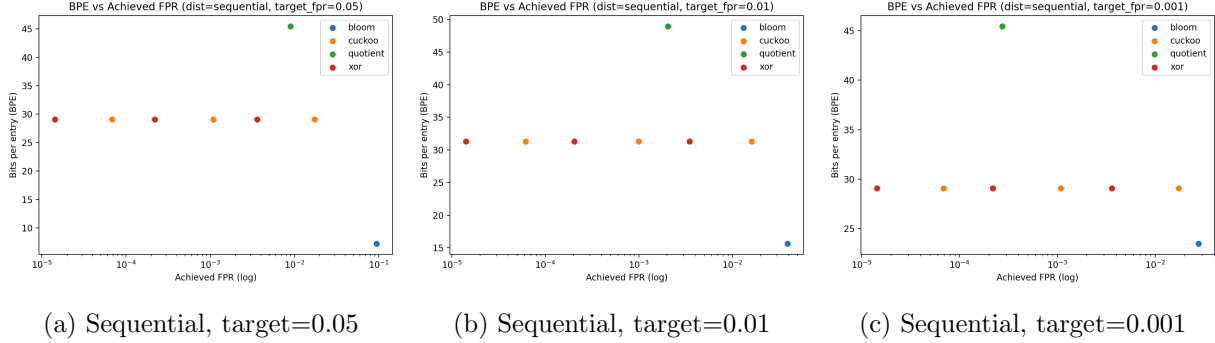


Figure 1: Space vs accuracy trade-off (sequential keys). Each dot is a tuned configuration.

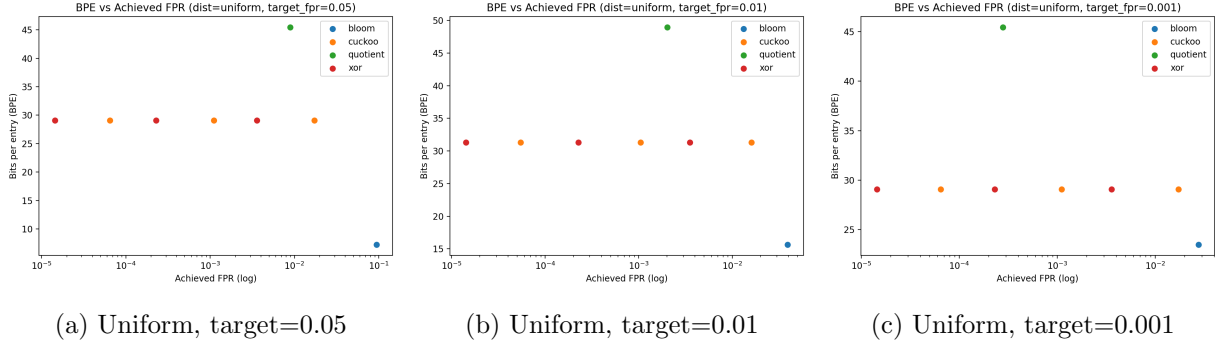


Figure 2: Space vs accuracy trade-off (uniform keys).

3.2 Insert/Delete Throughput vs Load Factor (Dynamic Structures)

Figures 3 and 4 show insert and delete throughput versus load factor for Cuckoo and Quotient filters.

Cuckoo filter throughput remains stable at moderate load factors but drops sharply near high occupancy. This is caused by increased eviction chains during insertion, which require relocating multiple fingerprints across buckets. At high load, the probability of reaching the maximum eviction bound increases, leading to insertion failures and occasional stash usage.

Quotient filter throughput degrades more gradually but consistently as load increases. This behavior is explained by cluster growth: as the table fills, insertions require shifting longer runs of entries to preserve quotient ordering. Average probe length and cluster size increase, which directly raises insertion cost.

XOR and Bloom do not appear in this experiment because XOR is static and Bloom does not support deletes without counting extensions.

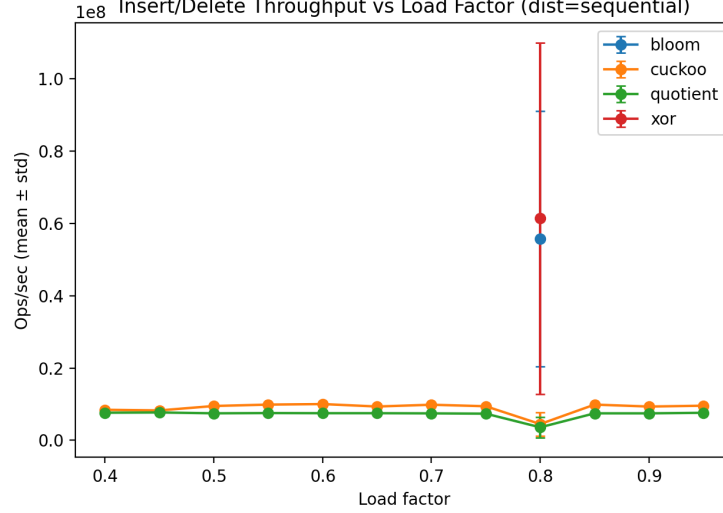


Figure 3: Insert/delete throughput vs load factor (sequential). Error bars show run-to-run variation.

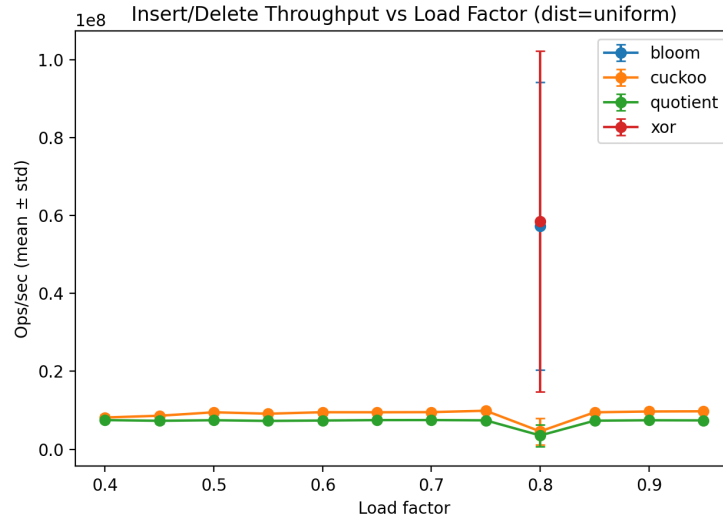


Figure 4: Insert/delete throughput vs load factor (uniform).

3.3 Lookup Throughput and Tail Latency vs Negative Lookup Share

Figures 5 and 6 plot lookup throughput and p95 latency versus negative lookup share (0

As the negative lookup share increases from 0% to 50%, throughput drops and tail latency rises across all structures. This occurs because the control flow alternates more frequently between hit and miss paths, increasing branch misprediction and instruction-path variability. This effect is confirmed by elevated branch-miss rates observed in perf counters (Table 1)

At 90% negative lookups, throughput recovers for Bloom, XOR, and Cuckoo filters. In this regime, the miss path dominates and becomes more predictable, reducing branch mispredictions and instruction-cache pressure. XOR benefits the most because its miss path is short and fixed.

Quotient filters maintain higher p95 latency even at high negative rates. Although misses terminate early, they still require scanning cluster metadata, which increases memory accesses and

cache pressure. This behavior aligns with higher L1-dcache and dTLB miss counts observed during these runs.

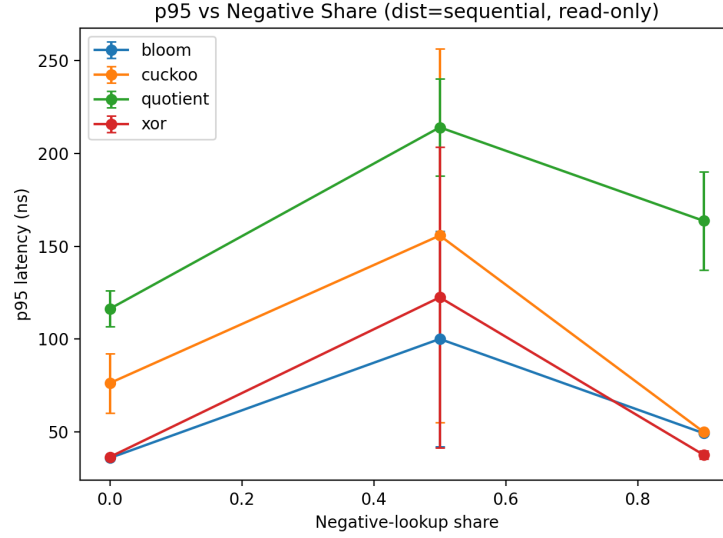


Figure 5: p95 latency vs negative lookup share (sequential, read-only).

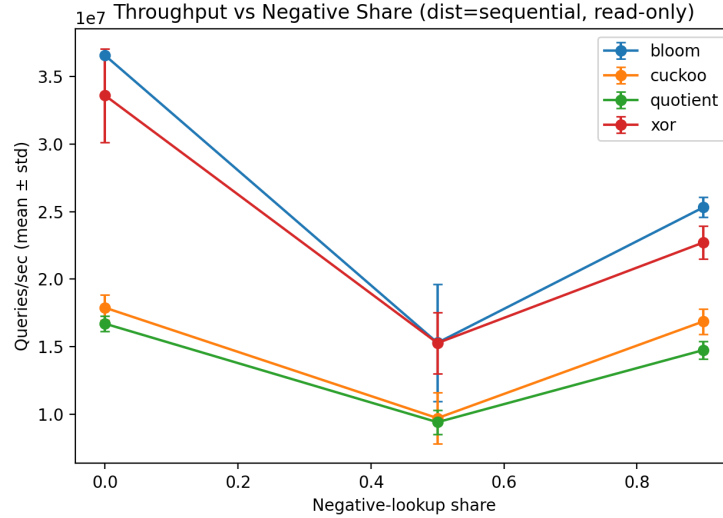


Figure 6: Throughput vs negative lookup share (sequential, read-only).

3.4 Thread Scaling and Concurrency Effects

Figure 7 reports throughput versus thread count (1–12) for balanced and read-mostly workloads.

Bloom and XOR scale well with thread count because their operations are read-only and involve minimal shared state. Bloom uses atomic bit operations, and XOR uses immutable arrays after construction, resulting in low contention.

Cuckoo and Quotient filters scale poorly beyond a small number of threads. Both rely on per-bucket or per-region locks to protect updates, and even read operations may contend on shared metadata. As thread count increases, lock contention and cache-line bouncing limit scalability. This effect is amplified under balanced workloads where inserts and deletes occur concurrently.

Perf counters support this explanation: higher cache-miss and dTLB-miss rates are observed at larger thread counts, indicating increased coherence traffic and memory system pressure.

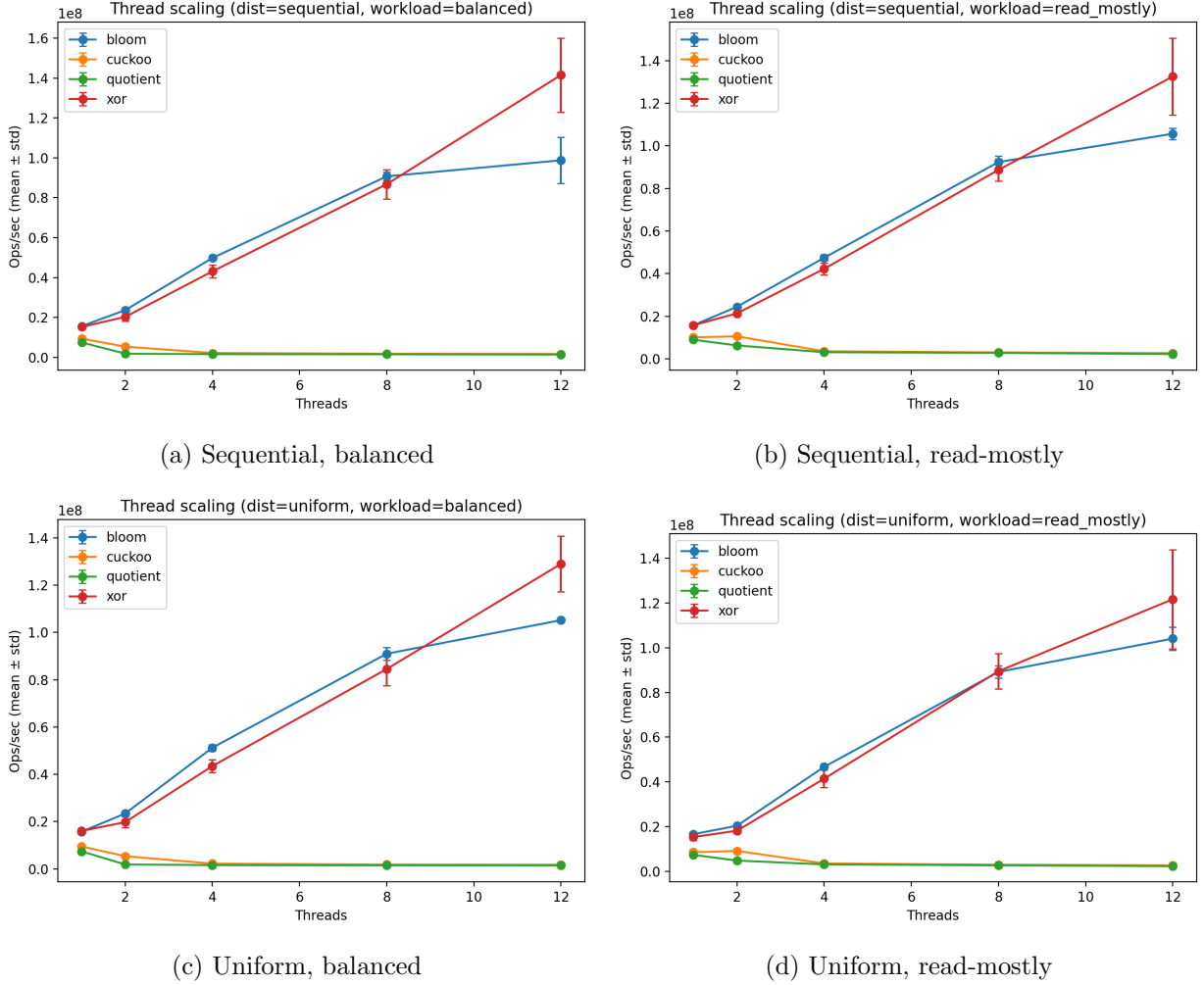


Figure 7: Thread scaling (mean $\pm$ std).

3.5 Perf Counters (Cache/TLB/Branch Evidence)

Table 1 shows a representative `perf stat` capture (uniform keys, 50% negatives, 1 thread). A non-trivial branch-miss rate and cache-miss rate matches the observed latency and throughput degradation at mixed hit/miss ratios. Some events are not counted on hybrid Intel CPUs for certain PMU domains, so both `cpu_core` and `cpu_atom` events may appear.

Metric	Value
cycles	946,146,884
instructions	953,458,290
branches	143,314,838
branch-misses	2,372,905 (1.66%)
cache-references	13,217,747
cache-misses	1,985,833 (15.02%)
dTLB-load-misses	1,608,489
L1-dcache-load-misses	4,973,509
LLC-load-misses	695,442
time elapsed	0.215647 s

Table 1: Representative perf counters (`perf.txt`).

4 Linking BPE, FPR, and Throughput to Structure Internals

The observed trade-offs are directly explained by structural design:

Bloom filters trade accuracy for minimal space and simple memory access.

XOR filters achieve excellent accuracy and throughput due to fixed fingerprints and no probing but require rebuilds.

Cuckoo filters balance space and accuracy with dynamic updates but suffer from eviction overhead at high load.

Quotient filters support dynamic operations with strong locality but incur higher space and latency due to cluster metadata and shifting operations.

Higher BPE generally reduces FPR but increases memory traffic, which can reduce throughput when cache capacity is exceeded.

5 Limitations, Anomalies, and Mitigations

Several limitations were observed:

XOR filters require full rebuilds on updates. This limits applicability to static or batch-update workloads. A mitigation is periodic rebuilds or hybrid designs combining XOR with a small dynamic overlay.

Cuckoo filters exhibit insertion failures and stash growth near high load factors. Lower target load factors or larger bucket sizes can mitigate this.

Quotient filters show long clusters that increase tail latency. Using larger quotient sizes or resizing earlier reduces cluster growth.

6 Conclusion

These results demonstrate that no single approximate membership structure dominates across all metrics. Space efficiency, accuracy, throughput, tail latency, and scalability are tightly coupled to internal design choices such as fingerprint size, probing strategy, and metadata layout. Understanding these trade-offs is essential for selecting the appropriate filter for a given workload and update pattern.